# Coordinate-wise SVRG for low signal-to-noise objectives

Kunal

One-page summary. Full details: `paper/technical_report.pdf`; every claim and its limits: `docs/CLAIMS.md`.

**What it is.** A variance-reduced optimizer with adaptive coordinate scaling, implemented in PyTorch and JAX, verified against an independent NumPy oracle, and studied on noisy finite-sum problems built to resemble tick data. The emphasis is on honest evidence: tuned baselines, held-out seeds, paired bootstrap intervals, and negative results reported as such.

**Contributions.**

1. **Three implementations that agree.** PyTorch, JAX and a NumPy oracle follow the same update rule, checked step for step by automated tests (including adversarial-input fuzzing); on a Colab T4 the PyTorch engine stayed within $1.8 \times 10^{-7}$ of the oracle over 100 steps.
2. **Short proofs.** Unbiasedness and a variance bound for the control-variate gradient; a staleness bound valid for the clipped adaptive engine; a numerical-safety property; and a corollary of the Johnson–Zhang theorem that ties SVRG's rate to the *preconditioned* condition number.
3. **A prediction from the theory, tested.** Jacobi-preconditioned SVRG with hyper-parameters read off the theorem and *no tuning* reached float64 round-off on 20/20 seeds of a badly scaled problem, needing 4.7× fewer gradient evaluations than the project's own adaptive engine, with measured per-epoch contraction 0.08 against the guaranteed 0.50.
4. **Fourteen experiments** against learning-rate-tuned Adam and SGD with momentum (constant and cosine-decay), plus ablations of the engine's invariants.

**Results at a glance** (synthetic data unless stated).

| Finding | Evidence |
|---|---|
| Variance reduction beats the best tuned baselines on well-scaled convex problems | 20/20 (least squares), 12/12 (logistic) seeds |
| Condition number of badly scaled features after Jacobi scaling | $1.1 \times 10^7 \to 82$ |
| Misaligned batches versus plain SGD (estimator error) | 2.1× *worse* |
| Textbook Adam on adversarial float32 inputs (extreme but finite) | fails in 76% of trials; full guard set 0% |
| Snapshot-interval sensitivity | within 2.1× of best for $16 \le K \le 256$ |

**What did not work, stated plainly.**

- **Real data:** on three small real datasets with raw features no method reached the target and Jacobi SVRG led the best tuned baseline by only 0.06–0.52 decades; after *standardising* the features, SVRG variants reached the target on 3/3 datasets and the baselines on 0/3, so the shortfall was raw-scale conditioning.
- **Scope:** offline research, not a production trading component. A compiled single-sample SVRG step costs 129 ns at $d = 32$ (inference dot product 15 ns), so the 2 ms GPU step is framework overhead, not the algorithm.
- **Market data:** a walk-forward study of 10 ETFs found **no tradable signal** (net Sharpe at best -0.76) and no optimizer effect (Jacobi SVRG reproduces OLS); regularisation mattered more.
- **GPU wall-clock:** on a T4, SVRG ties the best direct solve only at the largest size tested ($d = 2048$: 0.488 s against 0.490 s) and loses at smaller sizes; the engine step costs about 1.6 ms even with telemetry off.
- **No wall-clock win** over a direct solve at $n = 40000$, $d = 200$ (normal equations 0.06 s, SVRG 0.08 s); **no benefit** for walk-forward signal tracking (IC differences at most 0.0054).
- **Non-convex network:** sharper stationarity but worse held-out loss; with validation-tuned rates the deficit tracks adaptive scaling, not variance reduction. The adaptive engine has **no convergence proof**.

**Corrections made along the way** (logged in `CHANGELOG.md`): a claimed benefit of clamping over additive $\varepsilon$ was retracted after an ablation; a "best on all datasets" statement was corrected to a tie; a missing convexity hypothesis was added to a theorem statement.

**Verify it yourself.**

- Tests: `pip install -r requirements.txt && python -m pytest`
- All experiments: `python -m experiments.run_experiments`
- Report numbers match stored results: `python paper/make_numbers.py --check`